

WAB

Provadis School of International Management and Technology

**Performance of Various Python Runtimes for
a Bioinformatics Algorithm**

Testing the Smith-Waterman algorithm across CPython, PyPy,
and Python 3.14's native JIT

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Fortgeschrittene Programmierung
Reviewer: Prof. Dr. Jörg Daubert

28.04.2026

Abstract

Write your abstract here.

Contents

Abstract	II
List of Figures	IV
List of Tables	V
Glossary	VI
Abbreviations	VII
1. Introduction	1
1.1. Background	1
1.1.1. Cython	2
1.1.2. PyPy	2
1.1.3. CPython JIT Compiler	3
1.1.4. The Benchmarking Algorithm	3
1.2. Previous Work	5
1.3. Research Question	5
2. Methods	6
2.1. Design	6
2.2. Hardware Specifications	6
2.3. Python Runtimes and Tools	7
2.4. Dataset	7
3. Results	9
4. Discussion	10
5. Conclusion	11
References	i
AI Declaration	iii
Declaration of Authorship	vi

List of Figures

Figure 1	The Cython build pipeline, from a .pyx file to a compiled extension module.	2
Figure 2	Example of a local alignment between two DNA sequences using the Smith-Waterman algorithm. The final aligned subsequence is ACGTCG, with a score of 9.	4

List of Tables

Table 1	The runtimes included in the benchmark and their specifications.	7
Table 2	The additional (non-stdlib) Python packages used in the project. Only the primary packages are listed, not their dependencies. .	7
Table 3	Description of the synthetic scenario families used in the Smith-Waterman benchmark. p denotes the problem size.	8

Glossary

copy-and-patch: A JIT compilation strategy where optimized versions of certain functions are written in a lower-level language and then dynamically patched into the interpreter at runtime. (pp. 3)

local alignment: An alignment strategy that searches for the best matching subsections of two sequences instead of forcing an end-to-end alignment. (pp. 3)

warm-up phase: The initial execution period in which a JIT-enabled runtime collects profiling information and may compile hot paths before reaching steady-state performance. (pp. 3)

Abbreviations

DSL, Domain-Specific Language (pp. 3)
GCC, GNU Compiler Collection (pp. 2)
IDE, Integrated Development Environment (pp. 7)
JIT, Just-in-Time (pp. 1, 2, 3, 5)
MSVC, Microsoft Visual C++ (pp. 2, 7)
PEP, Python Enhancement Proposal (pp. 1)
SW, Smith-Waterman (pp. V, 3, 4, 5, 8)

1. Introduction

1.1. Background

Python¹ has emerged to be the most widely used programming language among many fields, including bioinformatics². It has many strengths, such as a very beginner friendly syntax, an exceptionally large ecosystem of packages, dynamic typing, and a feature rich standard library. One area where it is clearly lacking, however, is performance. This is due to the fact that the default implementation of Python, CPython, is interpreted and dynamically typed, which leads to significant overhead at runtime.

The Python interpreter does a lot of work to try to abstract away the underlying computing elements that are being used. At no point does a programmer need to worry about allocating memory for arrays, how to arrange that memory, or in what sequence it is being sent to the CPU. This is a benefit of Python, since it lets you focus on the algorithms that are being implemented. However, it comes at a huge performance cost.

— Gorelick M, Ozsvald I. ³

Many solutions have been proposed to address this issue while still remaining within the Python ecosystem. Chief among them are:

- Transpilation into C using Cython⁴
- Tracing Just-in-Time (JIT) compilation, as implemented in an alternative Python interpreter to CPython, called PyPy⁵
- Copy-and-patch JIT compilation, as proposed in PEP 744⁶, and with the latest stable implementation as of writing in Python 3.14⁷.

This is far from an exhaustive list. Many other approaches exist, such as the LLVM-based JIT compiler Numba⁸, the .NET-based IronPython⁹, the Python superset language Mojo¹⁰, etc., but due to the limited scope of this paper and due to the fact that these are either too specialized, not as frequently maintained, or not as mature as the ones mentioned above, they will not be included in this comparison.

1.1.1. Cython

Cython is a python module that allows users to write Python code interspersed with optional type annotations (in the form of `#cdef <variable>: <type>`). The file is then saved as a `.pyx` file and transpiled into C code using its *cythonize* toolchain. In the end, it creates a compiled module (a `.pyd` file on Windows, `.so` on Linux) using `setuptools` that can be imported and used from Python just like any regular module (see Figure 1). This naturally requires a compiler such as the GNU Compiler Collection (GCC) or Microsoft Visual C++ (MSVC) to be installed on the system⁴.

Cython build pipeline

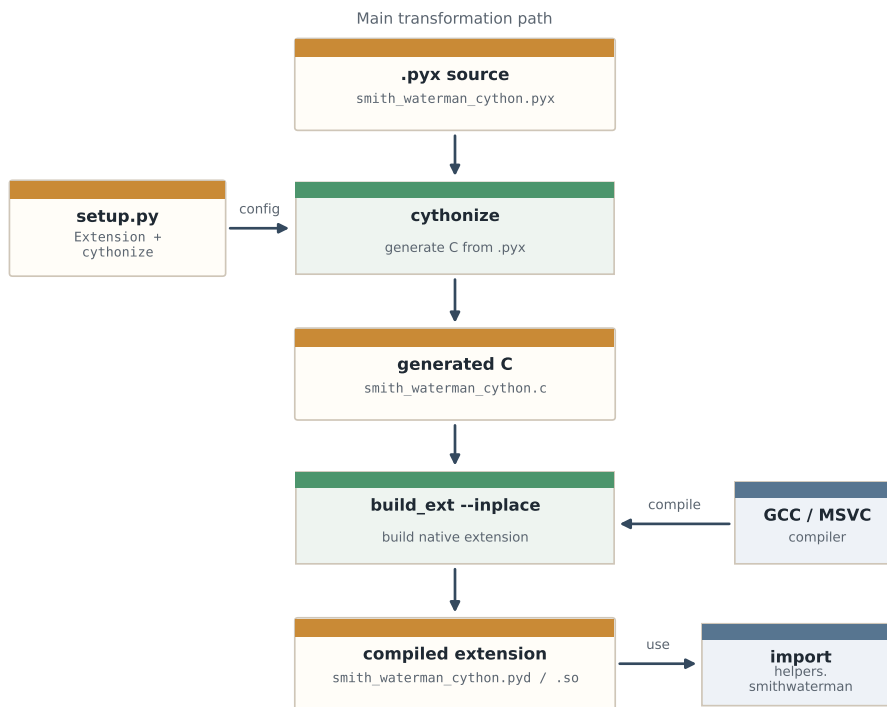


Figure 1: The Cython build pipeline, from a `.pyx` file to a compiled extension module.

1.1.2. PyPy

PyPy is an alternative implementation of the Python language, which features a tracing JIT compiler. It is a drop-in replacement for CPython, meaning that most Python code can be run on PyPy without modification.

PyPy runs on RPython¹¹, a subset of Python that is used to implement the PyPy interpreter itself. When a Python program is run on PyPy, the interpreter starts executing the code using an interpreter loop, similar to CPython. However, as it runs, it collects profiling information about which parts of the code are executed frequently (called “hot loops”). When it identifies such hot loops, it compiles them into native machine code using the tracing JIT compiler, which can lead to significant performance improvements for long-running programs after a warm-up phase¹².

The latest stable version of PyPy as of writing supports upto Python 3.11⁵, and according to the PyPy team, it is on average 2.9 times faster than CPython 3.11¹³.

1.1.3. CPython JIT Compiler

The CPython JIT is a copy-and-patch JIT compiler that was added in Python 3.14⁷. It speeds up frequently executed code by generating native machine code from prebuilt templates and patching these into the interpreter at runtime.

In a nutshell, copy-and-patch allows a high-quality template JIT compiler to be generated from the same DSL used to generate the rest of the interpreter.

— Bucher B, Ostrowski S. ⁶

1.1.4. The Benchmarking Algorithm

The Smith-Waterman (SW) algorithm¹⁴ is a local alignment algorithm that computes the highest-scoring matching subsequence(s) between two DNA/RNA/protein sequences, and is therefore a popular choice in bioinformatics. The scores are computed using dynamic programming, where a matrix of size $m * n$ is filled in based on the scores of neighboring cells, where m and n are the lengths of the two sequences being aligned. This involves assigning a score to each cell based on the scores of the top, left, and top-left neighboring cells, and the scoring scheme for matches, mismatches, and gaps (usually a +2 score for a match, −1 for a mismatch, and −2 for a gap. See Figure 2).

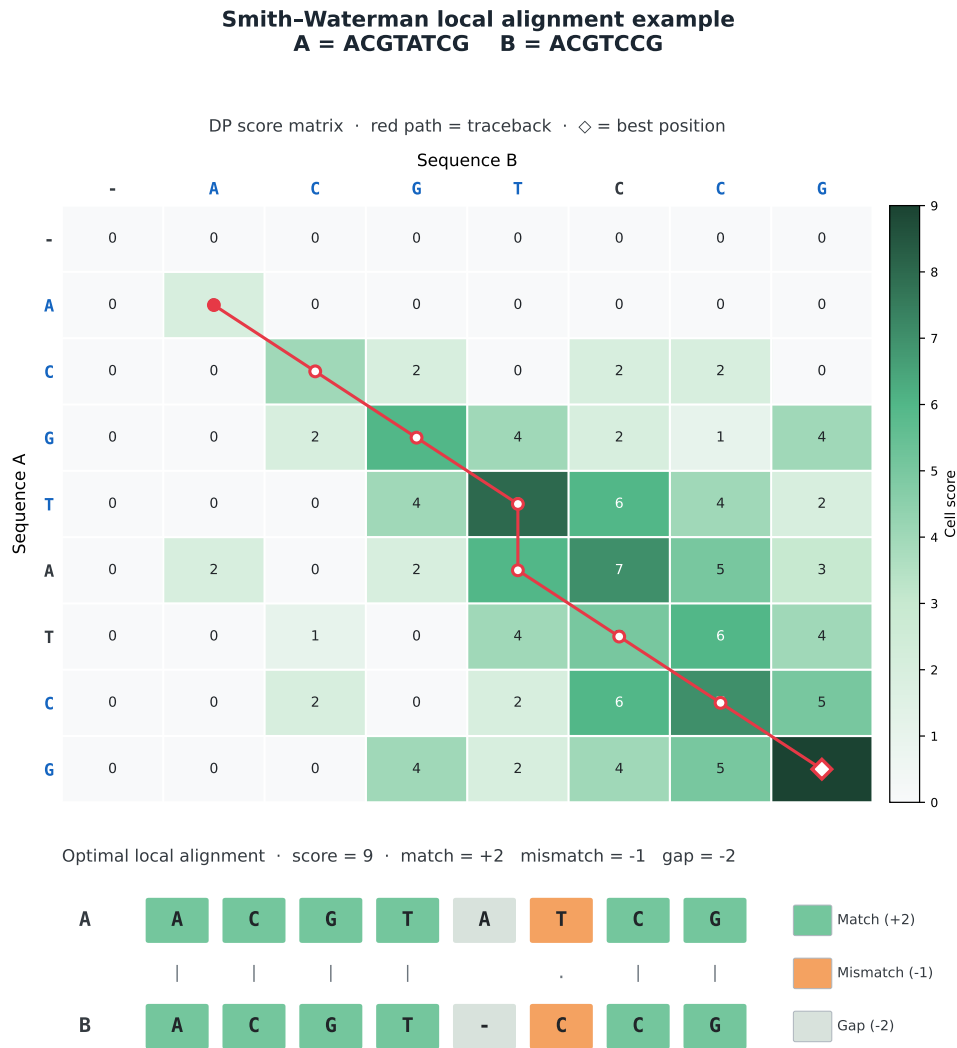


Figure 2: Example of a local alignment between two DNA sequences using the Smith-Waterman algorithm. The final aligned subsequence is ACGTCG, with a score of 9.

The SW algorithm has a time complexity of $O(m * n)$ for two sequences of length m and n because it compares each element of one sequence with each element of the other, so a naive implementation in Python is expected to perform very poorly. It nevertheless has two nested “hot loops”, which the performance-minded runtimes could potentially optimize. That, and the fact that it is still the best non-heuristic local alignment algorithm, and that it is relatively simple to implement (fewer than 100 lines of code), were the reasons why it was chosen as the algorithm for this performance comparison.

1.2. Previous Work

The PyPy team regularly benchmarks their implementation against CPython across a variety of workloads, and they have their results available on a dedicated section of their website¹³. However, they compare PyPy version 3.11 (the latest version of PyPy as of writing) to CPython 3.11. The CPython JIT was only added in Python 3.14, and therefore could not be included in their comparisons.

And although there have been many independent benchmarks comparing Cython and/or PyPy to CPython^{15,16}, the technology is advancing quite rapidly, so their findings may not represent the current state of affairs.

Additionally, no benchmarks have compared all the aforementioned runtimes to the new CPython JIT, as the project is still in its infancy. One of the maintainers of the new JIT compiler's optimizer (Ken Jin) has a few benchmarks where the performance of the JIT is compared to the standard CPython interpreter. The JIT seems to perform, on average, on par or slightly worse than the standard interpreter in these benchmarks¹⁷.

The Smith-Waterman algorithm specifically, does not seem to have been benchmarked across all these runtimes anywhere in the available literature.

1.3. Research Question

How does the SW algorithm perform across the different Python runtimes, namely the default CPython interpreter, the newer CPython JIT, PyPy, and Cython?

2. Methods

2.1. Design

The following strategies were used to ensure the benchmark was as fair as possible to each runtime, while playing to their individual strengths:

- Each runtime is given the same generated sequence pairs as input
- Each runtime is given the same number of warm-up iterations whether or not they'd theoretically benefit from them
- Each runtime was tested with problem sizes ranging from very short (~100 characters) to very long (~10,000 characters)
- The Cython version of the algorithm uses type annotations (`#cdef`) and was statically compiled (`/MT` flag) with default optimizations
- Each run is executed in a separate subprocess, but nevertheless run sequentially (no parallelism)
- The timing for each run was measured using the same method (`time.perf_counter_ns()`)
- Memory usage was not measured, as a method to do so that would be completely fair to all runtimes could not be found
- Every run was executed on the exact same hardware, on the same operating system (Windows 11 x64 version 25H2)

The benchmark code is available in the supplementary materials, with the entrypoint being `benchmark_sw.py`.

2.2. Hardware Specifications

The tests were run on a laptop with the following specifications:

- CPU model: Intel Core i5-10210U
- CPU cores: 4¹⁸
- CPU threads: 8¹⁸
- CPU base frequency: 1.60 GHz¹⁸
- CPU max turbo frequency: 4.20 GHz¹⁸
- RAM: 16 GB DDR4 2933 MT/s
- Storage: 512 GB NVMe SSD

To reduce interference, the tests were run with no other applications active apart from the IDE (VS Code), with the laptop connected to power, and with heavy background processes such as Windows Update disabled.

2.3. Python Runtimes and Tools

The test were performed on the following runtimes (Table 1):

Runtime	Version	Implementation Type
CPython	3.14.3	Interpreter (default Python runtime)
CPython JIT	3.14.3	Copy-and-patch JIT compiler
PyPy	7.3.20 with MSVC v.1941 (Python 3.11.13)	Tracing JIT compiler
Cython	3.2.4 with MSVC v.1944 (Python 3.14.3)	Transpiler to C

Table 1: The runtimes included in the benchmark and their specifications.

Additionally, the following Python packages were indispensable in the analysis and plotting of the results (Table 2):

Package	Version	Purpose
Pandas	3.0.1	Data manipulation and analysis
NumPy	2.4.2	Numerical computing
Matplotlib	3.10.8	Plotting and visualization
JupyterLab	4.5.5	Notebook environment for interactive analysis

Table 2: The additional (non-stdlib) Python packages used in the project.

Only the primary packages are listed, not their dependencies.

2.4. Dataset

The dataset was generated synthetically using the DNA alphabet (A, C, G, T) to cover a few common scenarios encountered in bioinformatics. The specific tested scenarios, and the rationale behind their choice, are described in Table 3.

Scenario Name	Length of A	Length of B	Description
Homologous region	p	p	Two mostly identical sequences that differ by a limited number of substitutions, as expected for homologous regions that diverged through mutation ¹⁹ .
Indel disruption	p	$p \pm \frac{p}{8}$	Homologous sequences that additionally contain a significant insertion or deletion event ¹⁹ .
Conserved motif	p	p	A short conserved element embedded in otherwise unrelated sequence context, as observed in interspersed repeats ²⁰ and transposons ²¹ .
Contained fragment	$\frac{p}{2}$	$\frac{p}{3} + \frac{p}{2} + \frac{p}{4}$	A shorter sequence contained within a longer one. The “needle in a haystack” scenario.
Random uniform	p	p	Unrelated sequences of the same length. Serves as a baseline.

Table 3: Description of the synthetic scenario families used in the Smith-Waterman benchmark. p denotes the problem size.

The sequences were generated using a custom script (available in the supplementary materials under `Code/helpers/case_generation.py`). To ensure reproducibility, the random seed was set to 42. The problem size p was tested at 100, 200, 500, 800, 1000, 2000, 5000, 8000, and 10,000 characters. For each problem size and scenario, 10 sequence pairs were generated, and each sequence pair was tested on each runtime 12 times (2 warmup runs and 10 timed runs) to account for variability at the system-level. The median was chosen as the summary statistic for the timed runs to account for outliers.

3. Results

Write your results here.

4. Discussion

Write your discussion here.

5. Conclusion

Write your conclusion here.

References

1. Van Rossum G, Drake FL. *Python 3 Reference Manual*. CreateSpace; 2009.
2. Mariano D, Ferreira M, Sousa BL, Santos LH, Melo-Minardi RC de. A Brief History of Bioinformatics Told by Data Visualization. In: Setubal JC, Silva WM, eds. *Advances in Bioinformatics and Computational Biology*. Springer International Publishing; 2020:235-246. doi:10.1007/978-3-030-65775-8_22
3. Gorelick M, Ozsvald I. *High Performance Python*. O'Reilly Media Inc.; 2025.
4. Behnel S, Bradshaw R, Citro C, Dalcin L, Seljebotn DS, Smith K. Cython: The Best of Both Worlds. *Computing in Science & Engineering*. 2011;13(2):31-39. doi:10.1109/MCSE.2010.118
5. The PyPy Team. PyPy: A Fast, Compliant Alternative Implementation of Python. Accessed April 27, 2026. <https://www.pypy.org/>
6. Bucher B, Ostrowski S. PEP 744 – JIT Compilation. 2024. Accessed March 10, 2026. <https://peps.python.org/pep-0744/>
7. Python Software Foundation. What's New In Python 3.14. 2025. Accessed March 10, 2026. <https://docs.python.org/3/whatsnew/3.14.html>
8. Lam SK, Pitrou A, Seibert S. Numba: a LLVM-based Python JIT compiler. In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. LLVM '15. Association for Computing Machinery; 2015. doi:10.1145/2833157.2833162
9. Foord M, Muirhead C. *Ironpython in Action*. Manning Publications Co.; 2009.
10. Mojo: Powerful CPU + GPU Programming. Accessed April 27, 2026. <https://modular.com/mojo>
11. The PyPy Team. The RPython Language. Accessed April 27, 2026. <https://rpython.readthedocs.io/en/latest/rpython.html>

12. The PyPy Team. PyPy: Goals and Architecture Overview. Accessed April 27, 2026. <https://doc.pypy.org/architecture.html#pypy-python-interpreter>
13. The PyPy Team. PyPy Speed Center. Accessed April 27, 2026. <https://speed.pypy.org/>
14. Smith TF, Waterman MS. Identification of common molecular subsequences. *Journal of Molecular Biology*. 1981;147(1):195-197. doi:10.1016/0022-2836(81)90087-5
15. Roghult A. Benchmarking Python Interpreters: Measuring Performance of CPython, Cython, Jython and PyPy. Published online 2016.
16. Zhong J, Hort M, Sarro F. Py2Cy: a genetic improvement tool to speed up python. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. GECCO '22. Association for Computing Machinery; 2022:1950-1955. doi:10.1145/3520304.3534037
17. Jin K. Reflections on 2 years of CPython's JIT Compiler: The good, the bad, the ugly. Accessed April 28, 2026. <https://fidget-spinner.github.io/posts/jit-reflections.html>
18. Intel Corporation. Intel[®] Core[™] i5-10210U Processor (6M Cache, up to 4.20 GHz) Specifications. 2019. Accessed February 26, 2026. <https://www.intel.com/content/www/us/en/products/sku/195436/intel-core-i510210u-processor-6m-cache-up-to-4-20-ghz/specifications.html>
19. Wheeler TJ, Eddy SR. nhmmer: DNA homology search with profile HMMs. *Bioinformatics*. 2013;29(19):2487-2489. doi:10.1093/bioinformatics/btt403
20. Jurka JW. Chapter Thirty-Nine - Approaches to Identification and Analysis of Interspersed Repetitive DNA Sequences. Adams MD, Fields C, Venter JC, eds. *Automated DNA Sequencing and Analysis*. Published online 1994:294-298. doi:<https://doi.org/10.1016/B978-0-08-092639-1.50043-5>
21. McClintock B. The origin and behavior of mutable loci in maize. *Proceedings of the National Academy of Sciences*. 1950;36(6):344-355. doi:10.1073/pnas.36.6.344

AI Declaration

The usage of AI tools within this project is documented here. I declare that I have documented all interactions with AI tools, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	I want you to rewrite my template from LaTeX to Typst, and make it look like it did before.	Recreated the template in Typst, maintaining the original appearance.
GitHub Copilot 2	I want you to write some tests Comparing the Python 3.14 JIT Compiler, PyPy, and Cython implementations of the Smith-Waterman algorithm.	Created a test suite that benchmarks the Python 3.14 JIT Compiler, PyPy, and Cython implementations of the Smith-Waterman algorithm.
GitHub Copilot 3	Put the test code in Code/, and use Python 3.15 instead of 3.14.	Added a runtime-generated Smith-Waterman validation and benchmarking suite in Code and retargeted the project text from Python 3.14 to Python 3.15.
GitHub Copilot 4	Okay, use python 3.14. 3.15 is still in pre-release. I've created the conda env named smith and installed the packages	Retargeted the Smith-Waterman suite and Exposé references back to Python 3.14 and validated the project against the configured environment.
GitHub Copilot 5	I think the warmup runs should be timed too. We can and should somehow separate it in the final summary though.	Modified the runtime worker to time each warmup repetition individually. Added a <code>per_run_times.csv</code> output that records every run's time with an <code>is_warmup</code> flag, keeping warmup and timed runs clearly separated in the data.

System	Prompt	Usage
GitHub Copilot 6	It'd also be interesting to see how the performance evolves across runs per algorithm, so make a plot of that.	Added a <code>run_convergence.pdf</code> plot showing runtime relative to each case's stable median across all run indices, with the warmup region shaded, faceted by problem size.
GitHub Copilot 7	The runtime distributions per case and algorithm would also be interesting to see, maybe as a box plot?	Added a <code>runtime_distributions.pdf</code> box plot showing the distribution of runtimes per case and algorithm, faceted by problem size.
GitHub Copilot 8	Add a power law fit of the runtime scaling with input size, and plot the empirical scaling exponent.	Added an empirical scaling exponent plot (<code>scaling_exponent.pdf</code>) fitting $t \propto n^\alpha$ via OLS in log-log space with 95% CI error bars, and a warmup penalty plot. The warmup penalty plot was subsequently removed for being too confusing to interpret.
GitHub Copilot 9	Move the run convergence plotting stuff to a new jupyter notebook. I might need to isolate that to singular scenarios too. Also move the scaling exponent plotting code in there.	Created <code>Code/plotting.ipynb</code> with interactive controls for both the run convergence and empirical scaling exponent plots. Both sections have filter controls for scenario, problem size, and algorithm, and the scaling exponent can be faceted per scenario to compare behaviour across input types.
GitHub Copilot 10	Help me create a python script that creates a figure to represent the Smith-Waterman algorithm's dynamic programming matrix and traceback path for a given pair of sequences.	Created <code>Code/swfigure.py</code> , which generates a PDF/PNG figure of the Smith-Waterman dynamic programming matrix.

System	Prompt	Usage
GitHub Copilot 11	Write a piece of Python code that can generate a figure that shows the steps in the Cython transpilation workflow. Call it cythonfigure.py and save the figure (PNG and PDF) to Code/Graphics.	[After a few iterations...] Created <code>Code/cythonfigure.py</code> , which generates a PDF/PNG figure of the Cython build pipeline from the <code>.pyx</code> source file to the compiled extension module.
GitHub Copilot 12	I screwed up my table formatting in the methods chapter, can you help me fix it?	Fixed the table formatting in the methods chapter by wrapping the table header and rows in a single <code>table()</code> call.

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, 28.04.2026

Rubin Chempananickal James